Tiling the TER L2 to L1 Merge

Measured prototype: 15.9x on the complete merge

Contents

Tiling the TER L2 to L1 Merge **1**
1. Summary . . . 1
2. The problem being solved . . . 2
3. Why the current implementation is memory-bound . . . 2
4. The structural fact that makes this unavoidable at scale . . . 2
5. The tiling idea . . . 3
6. Supporting transformations . . . 3
7. The algorithm . . . 4
8. Parameter selection . . . 5
9. Cost model . . . 5
10. Measured results . . . 5
11. Correctness . . . 6
12. Relationship to `ssp_oracle.cpp` . . . 6
13. Consequence for the dense-NTT verdict . . . 7
14. Proposed implementation in this repository . . . 7
15. What has not been measured . . . 7
16. Reproduction . . . 8

Tiling the TER L2 to L1 Merge

Status: measured prototype, not wired into the repository. **Scope:** the arithmetic-counting stage of the L2 to L1 merge, plus its interaction with the existing structural validation. **Headline:** 15.9x on the complete merge at the largest executed paper-scaling point, with the raw triple count, the structurally valid triple count and the unique parent set all unchanged.

---

1. Summary

TER's L2 to L1 merge spends 97-99% of its time in one loop: scan every pair `(a, b)` from lists A and B, compute the quotient residue that a matching `c` would need, and ask a lookup table how many such `c` exist. That table has one slot per residue, so it is `Q` entries wide.

Under the classical TER paper scaling, `Q` is approximately `L^2 / lambda` with `lambda` held near 2-4. So the table is quadratic in the list size: 256 MiB at `L = 16384`. Every one of the `L^2 = 268` million probes is an effectively random access into that 256 MiB table, and almost every one misses cache.

Tiling fixes this without skipping any work. Bucket A and B by the high bits of their quotient. If `a` lies in bucket `i` and `b` in bucket `j`, then `a + b` is confined to a window of width `2S` that depends only on `i + j`. Group all bucket pairs by `m = i + j` and process them together, holding only that one `2S`-wide slice of the table. Sized so the slice is 256 KiB, every probe becomes an L2 cache hit.

Same number of operations, same answers, reordered to respect the memory hierarchy.

2. The problem being solved

The L2 to L1 merge takes three lists A, B, C of level-2 ternary vectors and must find every triple `(a, b, c)` that is simultaneously:

1. **arithmetically admissible** - `q_a + q_b + q_c === tau (mod Q)`, where `q_x` is the entry's quotient residue and `Q = 2^(r1 - r2)`;
2. **structurally valid** - the coordinatewise sum `a + b + c` stays inside `{-1, 0, +1}`;
3. **distribution correct** - the resulting parent vector has exactly the level-1 digit counts.

Condition 1 is the cheap filter applied to all `L^2` pairs. Conditions 2 and 3 are the expensive per-candidate checks, applied only to survivors.

The existing production-shaped comparator (`direct_production_method` in `src/q3report_paper_crossover.cpp`) implements condition 1 as:

```
build offsets[0 .. Q]                  // counting-sort index over C by quotient
for each a in A:
    base = (tau - q_a) mod Q
    for each b in B:
        required = (base - q_b) mod Q
        raw += offsets[required + 1] - offsets[required]
```

This is `O(L^2)` time and `O(Q + L)` memory, and it is exactly right as an algorithm. The problem is entirely in where the memory lands.

3. Why the current implementation is memory-bound

Measured on the target machine (single thread, 64 MiB working set):

quantity	value
streaming read+write bandwidth	43.15 GB/s
`DIRECT_PRODUCTION` at `L = 16384`	2199.8 ms for 268.4 M probes
effective cost per probe	approximately 8.2 ns

Eight nanoseconds per probe is DRAM-latency territory, not arithmetic. The inner loop performs two subtractions, a mask, two loads and an add. On a modern core that is sub-nanosecond work if the loads hit cache. They do not: the probe index `(tau - q_a - q_b) mod Q` is essentially uniform over `[0, Q)`, and `Q * 4` bytes is 256 MiB against a few MiB of last-level cache.

4. The structural fact that makes this unavoidable at scale

This is not an artifact of one benchmark point. The TER paper scaling ties `Q` to `L`:

```
lambda = |A| * |B| / Q,  held near constant (2, then 4, across the declared ladder)
  =>  Q ~ L^2 / lambda
```

So the lookup table is **quadratic in the list size**. It cannot be made to fit in cache by choosing a nicer constant; it grows exactly as fast as the pair space it indexes.

L	q_bits	Q	table size
2048	21	2 097 152	8 MiB

L	q_bits	Q	table size
4096	23	8 388 608	32 MiB
8192	24	16 777 216	64 MiB
16384	26	67 108 864	256 MiB

The table crosses out of cache somewhere between `L = 2048` and `L = 4096`, which is precisely where the measured speedup of the tiled version jumps (Section 10).

---

5. The tiling idea

5.1 Core insight

Chop the residue range `[0, Q)` into `P` bins of width `S`, so `Q = P * S`. Place each `a` in bin `a / S` and each `b` in bin `b / S`. Then:

> If `a` is in bin `i` and `b` is in bin `j`, then `a + b` lies in `[(i + j) * S, (i + j + 2) * S)`.

That window is only `2S` wide, and **it depends only on `i + j`**, not on `i` and `j` individually. Every bucket pair sharing the same `m = i + j` produces sums landing in the same narrow window.

5.2 Worked example

With `S = 100`:

```
a in bin 3 (300-399)  +  b in bin 5 (500-599)   ->  a+b in [800, 1000)
a in bin 2 (200-299)  +  b in bin 6 (600-699)   ->  a+b in [800, 1000)
a in bin 4 (400-499)  +  b in bin 4 (400-499)   ->  a+b in [800, 1000)
a in bin 1 (100-199)  +  b in bin 7 (700-799)   ->  a+b in [800, 1000)
a in bin 0 (  0- 99)  +  b in bin 8 (800-899)   ->  a+b in [800, 1000)
```

All five bucket pairs have `m = 8`. All their sums land in `[800, 1000)`. Process them as one group while holding only slots 800-999 of the table.

5.3 Why this beats sorting the pairs

The naive way to get locality would be to sort the `L^2` pair sums. That is `L^2 log L` work and `L^2` memory - strictly worse than the problem it fixes. Tiling gets the same locality by sorting the **inputs** (`O(L)` counting sort on each of A and B) and letting the pairs inherit the ordering for free.

---

6. Supporting transformations

Three small changes remove all remaining per-probe overhead.

6.1 Re-key C by `(tau - q_c)`

Instead of computing "which `c` do I need?" inside the hot loop, precompute once per `c`:

```
key_c = (tau - q_c) mod Q
```

The condition `q_a + q_b + q_c === tau` becomes simply `q_a + q_b == key_c`. The inner loop is then a bare addition - no subtraction against `tau`, no per-pair modular fixup.

6.2 Double the keys to eliminate the modulo

`q_a + q_b` ranges over `[0, 2Q)` and can wrap. Rather than masking in the inner loop, store each C key twice - once at `k` and once at `k + Q`:

```
dk = sorted multiset { key_c, key_c + Q : c in C }        // 2|C| entries
```

Plain addition now always lands correctly; wraparound is handled by construction. Because `dk` is sorted, a single monotone cursor walks it across all windows for the whole run.

6.3 Clear only what was touched

A 2S-wide window contains only about `2S * |C| / Q` C-entries - roughly 16 at `L = 16384`. So after finishing a window, zero those approximately 16 slots rather than wiping the 256 KiB array. Total clearing cost across the entire run is `O(|C|)`, not `O(Q)`.

---

7. The algorithm

```
input : A, B, C (quotient residues in [0,Q)), tau, Q = 2^q_bits
output: raw = number of (a,b,c) with q_a + q_b + q_c === tau (mod Q)

S  <- 1 << SB                        # window half-width, cache-sized
P  <- Q / S                          # number of buckets

# ---- setup, O(L log L) ----
dk <- sort({ (tau - q_c) mod Q } union { (tau - q_c) mod Q + Q })      # 2|C| entries
(Ao, Ag) <- counting_sort_by_bucket(A, SB)     # Ao values, Ag bucket offsets
(Bo, Bg) <- counting_sort_by_bucket(B, SB)

cnt[0 .. 2S) <- 0
lo  <- 0
raw <- 0

# ---- main pass ----
for m in 0 .. 2P-2:
    w0 <- m * S ;  w1 <- w0 + 2S

    advance lo while dk[lo] < w0                        # monotone cursor, never rewinds
    hi <- lo
    while dk[hi] < w1:                                  # scatter this window's C keys
        cnt[dk[hi] - w0] += 1 ;  hi += 1

    if hi > lo:                                         # window is non-empty
        for i in max(0, m-P+1) .. min(P-1, m):
            j <- m - i
            for a in Ao[Ag[i] .. Ag[i+1]):
                for b in Bo[Bg[j] .. Bg[j+1]):
                    raw += cnt[a + b - w0]              # always an L2 hit

    for x in lo .. hi:                                  # selective clear
        cnt[dk[x] - w0] <- 0

return raw
```

The bounds `max(0, m-P+1) .. min(P-1, m)` enumerate exactly the bucket pairs `(i, j)` with `i + j = m` and both indices in range.

8. Parameter selection

`S` is the only tuning knob. It trades window count against window size:

- the live slice is `2S * 4` bytes and must fit comfortably in L2;
- the number of windows is `2P - 1 = 2Q/S - 1`, and each carries fixed loop overhead;
- the number of bucket pairs visited is about `P^2`, each covering about `(L/P)^2` real pairs, so `P` must stay small enough that per-bucket-pair overhead does not dominate.

The prototype uses `SB = 15`:

derived quantity	value at `Q = 2^26`
`S`	32 768
live slice `2S * 4 B`	256 KiB
`P = Q / S`	2 048
windows `2P - 1`	4 095
entries per A/B bucket	approximately 8
C keys per window	approximately 16

This was not swept exhaustively; `SB = 15` was chosen to put the slice in L2 and worked at every point tested. A sweep is listed as open work in Section 15.

9. Cost model

	current	tiled
pair probes	`L^2`	`L^2` (identical)
index construction	`O(Q + L)`	`O(L log L + L)`
clearing	`O(Q)` once	`O(L)` total
live working set	`Theta(Q) = Theta(L^2)`	`Theta(L + S)`
probe locality	random over `Q`	sequential within `2S`

Nothing is skipped and no candidate is pruned. The asymptotic time is unchanged at `Theta(L^2)`. The entire gain is the constant factor between an L2 hit and a DRAM round trip.

10. Measured results

10.1 Stage in isolation (standalone prototype, uniform random quotients)

q_bits	L	lambda	current	tiled	speedup	peak memory
23	4 096	2.0	81.7 ms	6.9 ms	11.9x	32 MiB -> 0.3 MiB
24	8 192	4.0	373.5 ms	26.3 ms	14.2x	64 MiB -> 0.4 MiB
26	16 384	4.0	1951.4 ms	149.8 ms	13.0x	256 MiB -> 0.6 MiB

Raw triple counts identical at every point.

10.2 Complete merge on real snapshots

Built with `build_paper_scaling_snapshot` (seed `0x1001`), then arithmetic scan plus the repository's own `ter::audit_candidate` (structural, parent construction, distribution) plus deduplication into the parent set.

L	Q	lambda	arith now	arith tiled	structural	dedup	**merge now**	**merge tiled**	**speedup**
2 048	2^21	2.0	9.1 ms	1.5 ms	0.7 ms	0.3 ms	10.1 ms	2.5 ms	**4.08x**
4 096	2^23	2.0	92.8 ms	5.8 ms	1.4 ms	0.6 ms	94.8 ms	7.8 ms	**12.10x**
8 192	2^24	4.0	398.4 ms	22.1 ms	6.7 ms	2.3 ms	407.4 ms	31.0 ms	**13.13x**
16 384	2^26	4.0	2199.8 ms	120.0 ms	14.5 ms	5.1 ms	2219.4 ms	139.6 ms	**15.90x**

10.3 Why the whole-merge figure tracks the stage figure

L	arithmetic share of merge
2 048	90.1%
4 096	97.9%
8 192	97.8%
16 384	**99.1%**

Amdahl's law barely applies because `lambda` is small. At `L = 16384` the scan visits 268.4 M pairs and only 65 524 survive to structural validation - a 4 096:1 filter. `audit_candidate` runs 65 524 times for 14.5 ms against 2199.8 ms of arithmetic.

The trend runs the right way: 4.08x -> 15.90x as `L` grows, because the arithmetic share rises toward 100% and the table falls further out of cache. `L = 2048` is the weak point precisely because `Q = 2^21` is only 8 MiB and already partly cache-resident.

---

11. Correctness

The transformation is a pure reordering. Three independent checks were run at every point:

1. **Raw triple count** - tiled scan equals the shipping scan exactly (65 524 at `L = 16384`, 32 374 at `L = 8192`).
2. **Structural outcome** - running `ter::audit_candidate` over the surviving triples yields 22 528 valid triples and 22 485 unique parents at `L = 16384`, unchanged.
3. **Completeness by construction** - every `(a, b)` pair belongs to exactly one bucket pair `(i, j)`, hence exactly one group `m = i + j`. Groups partition the pair space, so no pair is visited twice and none is missed. The key doubling guarantees that a sum which wraps modulo `Q` still matches, because both `k` and `k + Q` are present in `dk`.

---

12. Relationship to `ssp_oracle.cpp`

The uploaded oracle contains two distinct streaming techniques. They map onto TER differently.

Pair streaming (`SsPairStream`, heap-based k-way merge, ascending AB against descending CD). This buys memory: it walks an `L^2` pair space in `O(L)` storage without materializing it. TER's merge already gets that from its nested loop, and the heap costs `O(log)` per step where TER pays `O(1)`. **Not applicable to the merge.**

Stream probing (`plan_parallel_stream_probe`, prefix-code partitioned binary enumeration). This is for a side too large to materialize at all. That is not the L2 to L1 merge, where the lists hold only about 16 K entries. It **is** the right shape for TER's L3 generation, where the half-vector domain saturates beyond `2^31` for `shape_n >= 252` and `generate_level3_lists` currently returns `ResourceLimit`. The prefix-code decomposition - a worker derives its own start in `O(prefix_bits)` adds, no shared generator, no replay - is directly transferable there.

The tile dispatcher is the concept this document borrows: process in chunks sized to fast memory rather than striding the whole domain.

---

13. Consequence for the dense-NTT verdict

The dense NTT screening experiment recorded `DENSE NTT NO-GO`. Tiling makes that more decisive, not less, because it accelerates the winning side.

At `L = 16384`, after the Gentleman-Sande / Cooley-Tukey rework of the NTT kernel:

comparator	time	NTT amortized / comparator
`DIRECT_PRODUCTION` as shipped	2199.8 ms	5.2x
complete merge, tiled	139.6 ms	**81x**

This retroactively justifies a caution recorded during the NTT work: optimizing the transform while leaving the baseline untuned was flattering the NTT. The baseline had a 13-16x win sitting unused.

The structural argument is untouched. Direct counting is `Theta(L^2) = Theta(Q)`; dense NTT is `Theta(Q log Q)`. Tiling improves a constant; it does not alter the `log Q` separation.

---

14. Proposed implementation in this repository

Do not replace `direct_production_method`. It is the declared comparator of a completed experiment with a recorded verdict and a committed CSV. Replacing it silently would break comparability with every row already recorded.

Proposed shape:

1. Add `direct_production_tiled_method` alongside the existing two methods in `src/q3report_paper_crossover.cpp`, with its own `PaperMethodOutcome`, its own independent memory preflight, and its own component timings.
2. Extend Phase A (correctness, untimed) to assert the tiled raw triple count equals both the existing `DIRECT_PRODUCTION` count and the selected-target sum over the NTT coefficients.
3. Add CSV columns `direct_tiled_total_median_ns`, `direct_tiled_peak_bytes`, `ntt_over_direct_tiled`, leaving all existing columns untouched.
4. Add deterministic tests: window partition completeness, wraparound at `0` and `Q-1`, agreement with `direct_full_pair_counts` at several `S` values, and behaviour when a window is empty.
5. Run the declared seed and repetition schedule, and report as an additional row family rather than a revision of the existing results.

---

15. What has not been measured

Stated explicitly so none of the above is over-read:

- **`S` was not swept.** `SB = 15` was picked to fit L2 and never tuned. The optimum may differ per `Q`, and small `Q` may prefer no tiling at all.
- **Only the L2 to L1 merge was measured**, not a complete `solve_full_ter` attempt, which also performs L3 to L2 generation, three merge groups and witness verification. The share of a whole attempt is unknown.
- **Section 10.1 used synthetic uniform quotients.** Section 10.2 used real snapshots, and the two agree, but the synthetic table should be read as corroboration only.
- **Single seed** (`0x1001`) for the real-snapshot measurements. The declared multi-seed schedule has not been run against the tiled path.
- **Single thread throughout.** No claim is made about parallel behaviour, although the window decomposition is embarrassingly parallel over `m`.

- **No prototype is in the repository.** All figures come from standalone programs linked against the repository library.

---

16. Reproduction

Measurements were taken on an Intel Core Ultra 9 275HX, single-threaded, GCC 11.4, `-O3 -march=native`, with the repository library built from the same tree.

- Bandwidth figure: streaming read-modify-write over a 64 MiB `uint32` array, 10 repetitions after 2 warm-up passes.
- Stage figures: standalone prototype, both methods in one process, results cross-checked.
- Merge figures: `build_paper_scaling_snapshot` with seed `0x1001`, quotients extracted from the audited snapshot, both scans timed in-process, then `ter::audit_candidate` timed over the collected triples, then dedup into a `std::set<ReferenceTernaryVector>` timed separately.

`cmake` is unavailable in the environment where these were produced; builds used a direct `g++` invocation mirroring the `CMakeLists.txt` configuration.